Tasks on docker-01

1. Setup a virtual machine using Ubuntu image.

Implementation Steps:

1. Install Oracle VM VirtualBox

2. Download the Ubuntu ISO image

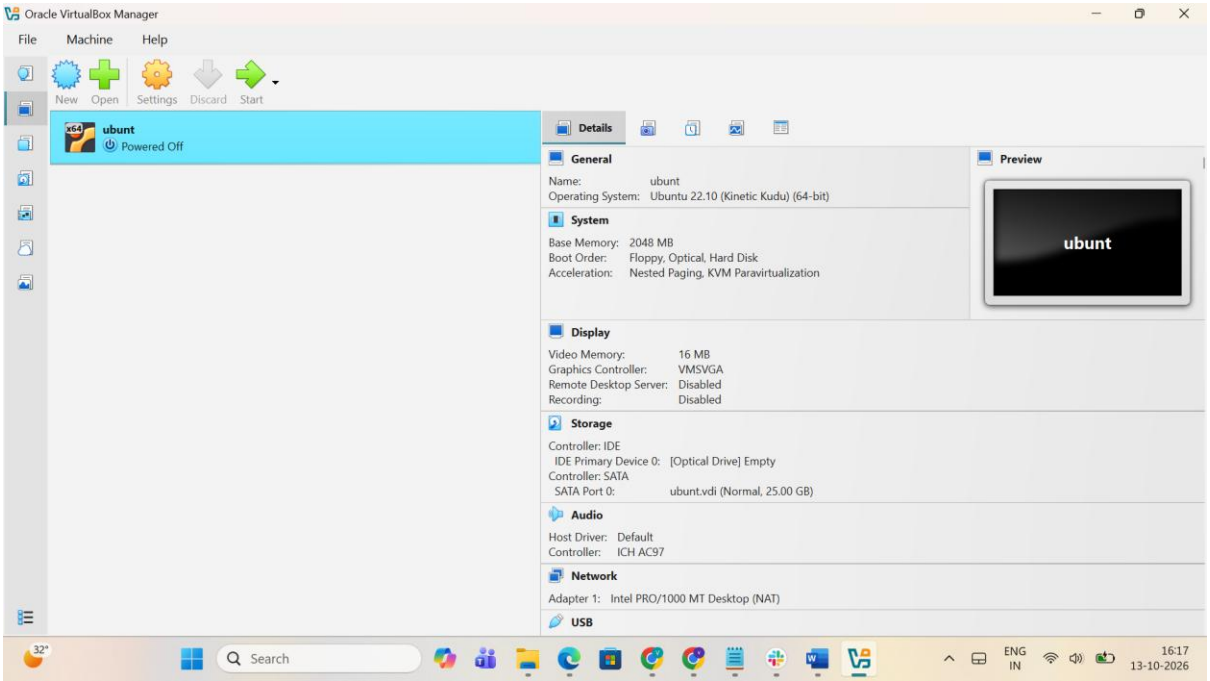
3. Create a new virtual machine

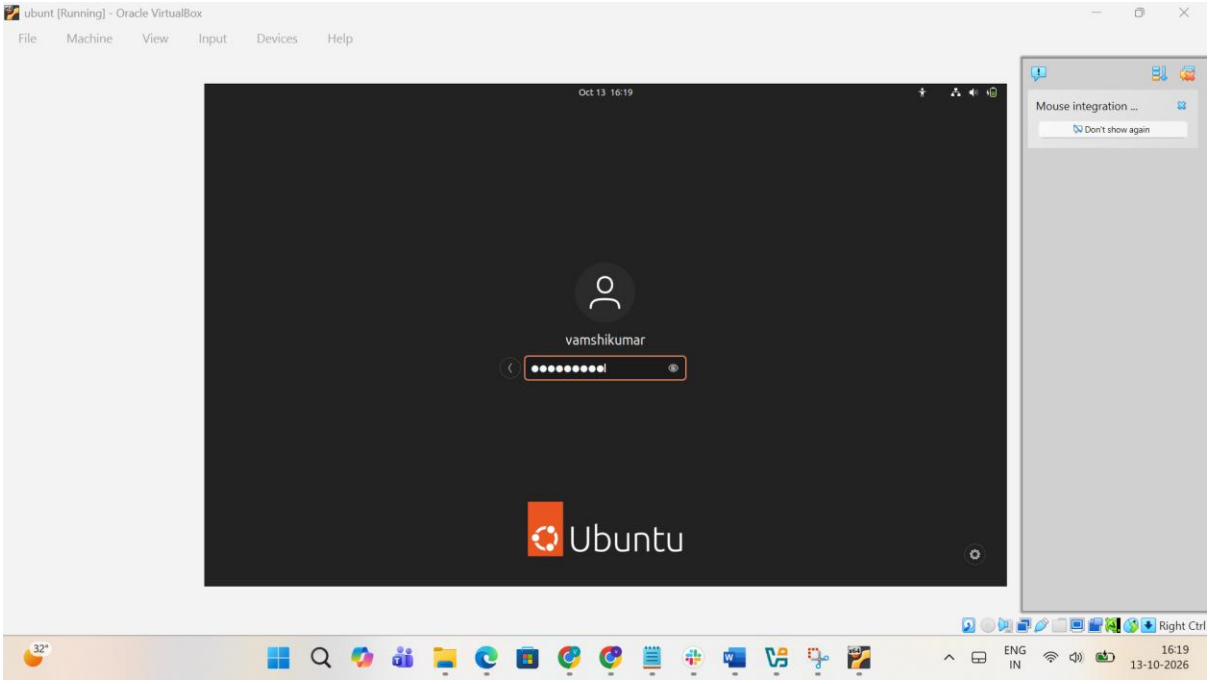
    1. Enter the following details:
    2. Name: Ubunt-VM-22
    3. Type: Linux
    4. Version: Ubuntu (64-bit)
    5. ISO Image: Select the downloaded Ubuntu ISO file.
    6. Configure the hardware resources. For example:
    7. RAM: 8 GB (8192 MB), if available.
    8. Processors: 2 CPUs, if available.
    9. Create a virtual hard disk:
    10. Disk size: 25 GB or more.
    11. Use the default virtual disk format unless your training requires otherwise.
    12. Click Finish.

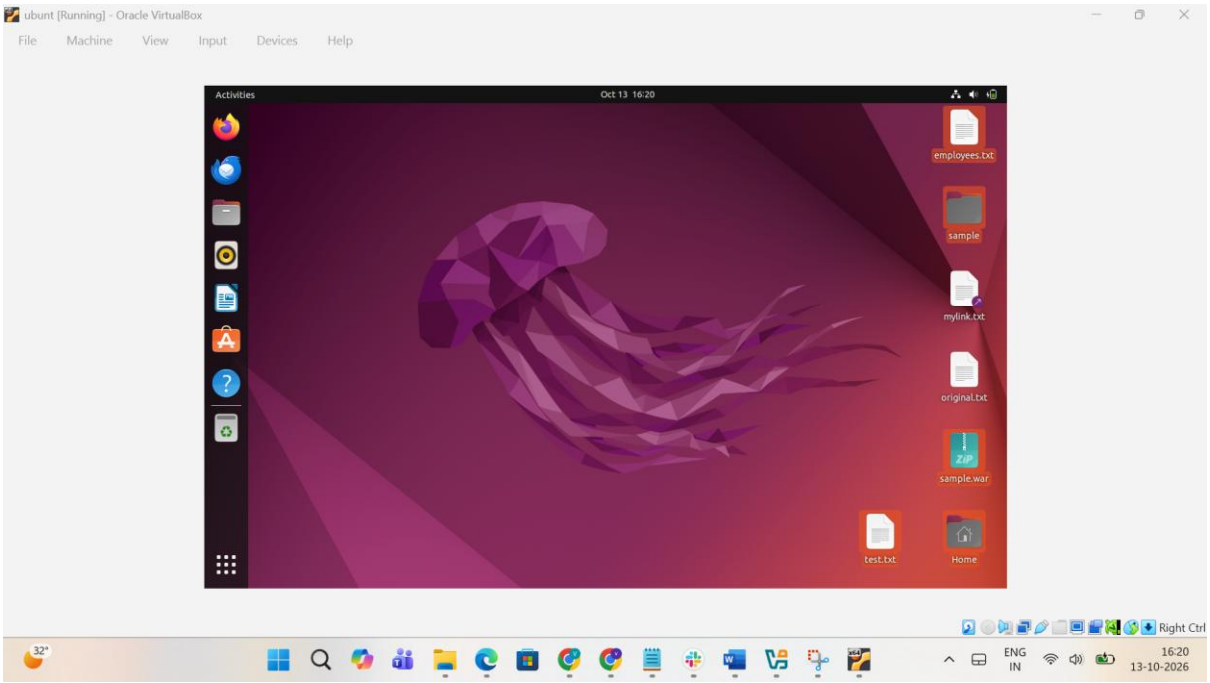
4. Start the virtual machine

5. Install Ubuntu

6. Log in to Ubuntu







2. Install docker on ec2.

Implementation steps

1. Launch an EC2 instance using the Amazon Linux 2023 AMI.

2. Connect to the instance using EC2 Instance Connect or SSH.

3. Update the system packages. sudo dnf update -y

4. Install Docker. sudo dnf install -y docker

5. Start the Docker service. sudo systemctl start docker

6. Enable Docker to start automatically when the instance reboots. sudo systemctl enable docker

7. Verify the Docker service. sudo systemctl status docker --no-pager

8. Check the Docker version. sudo docker --version

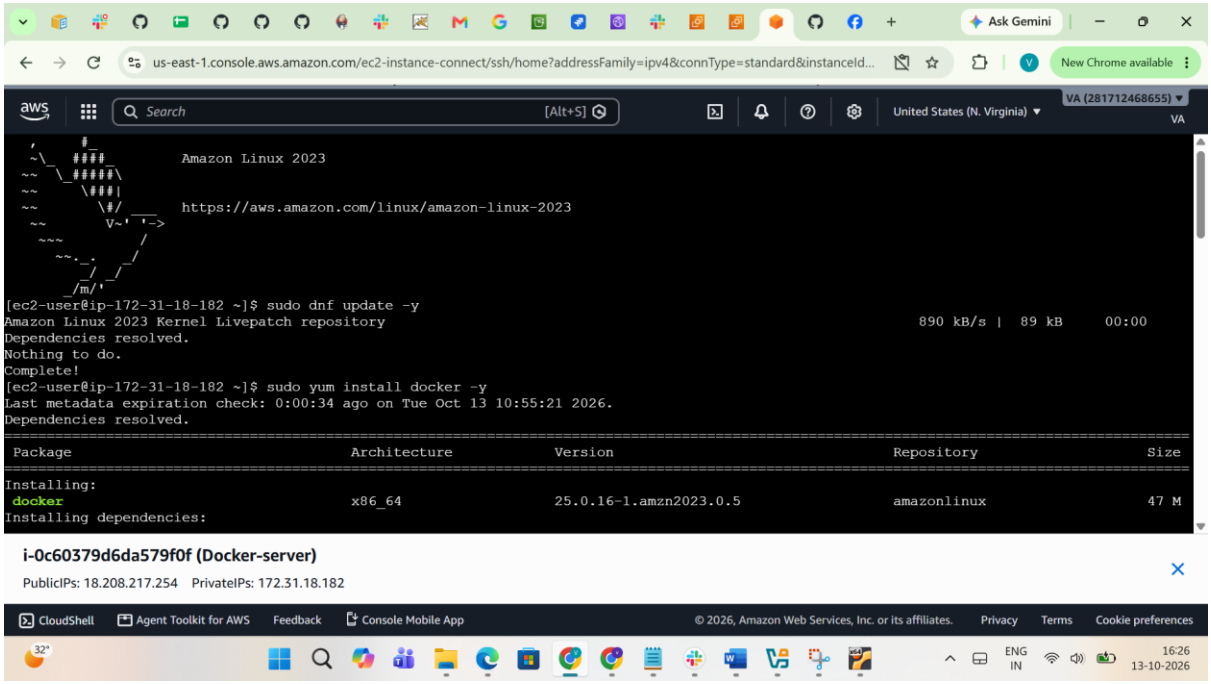
9. Run a test container. sudo docker run hello-world

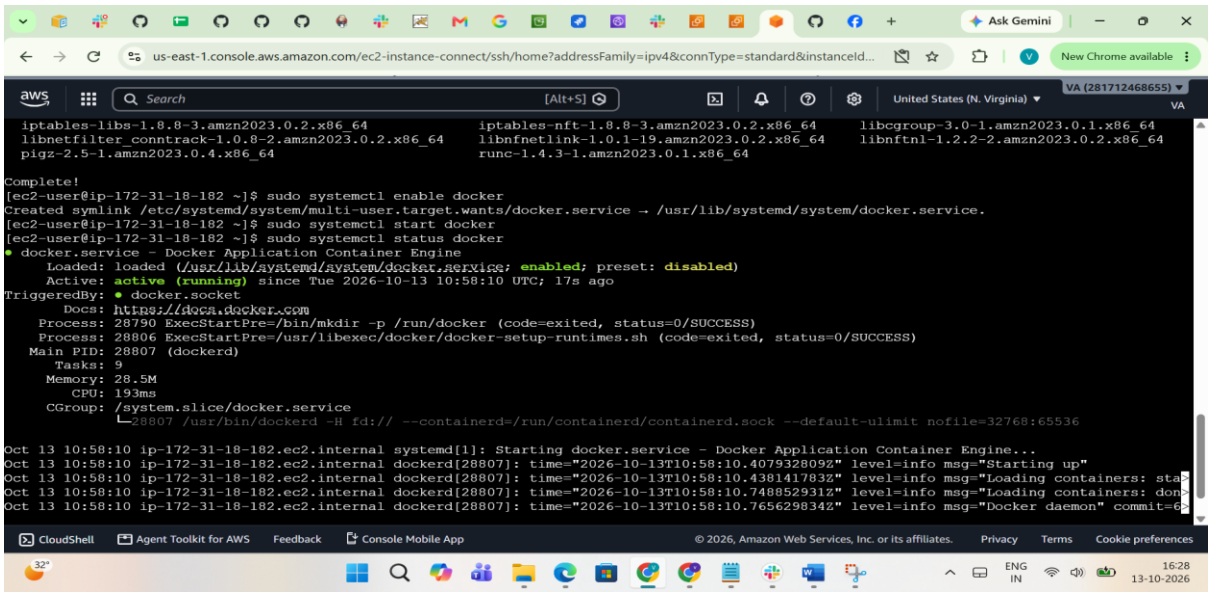
Expected result

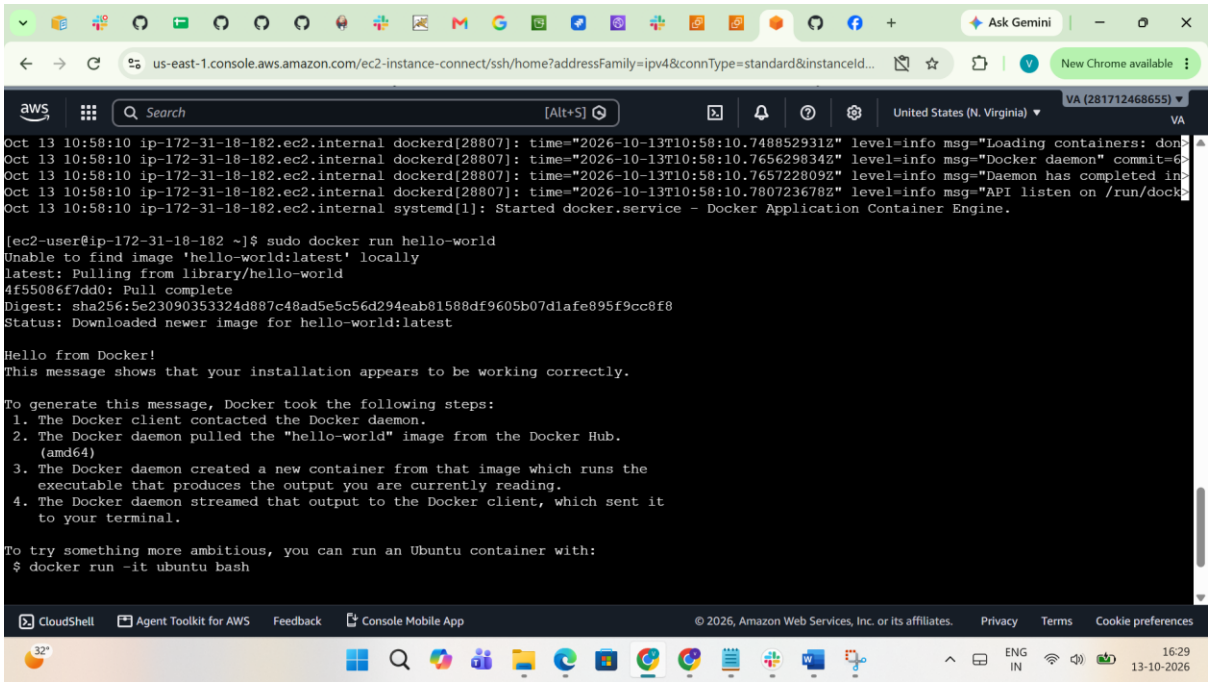
• Docker is installed successfully.

• The Docker service shows active (running).

• The test container displays Hello from Docker!.









3. Pull 5 docker images. (nginx, Apache tomcat, ubuntu, Jenkins, SonarQube)

Implementation steps

Run the following commands one at a time.

1. Pull Nginx
   sudo docker pull nginx:stable
2. Pull Apache Tomcat
   sudo docker pull tomcat
3. Pull Ubuntu
   sudo docker pull ubuntu:24.04
4. Pull Jenkins Your original image was jenkins:2.60.3:
   sudo docker pull jenkins:2.60.3
5. Pull SonarQube
   sudo docker pull sonarqube:community
6. Verify the downloaded images
   sudo docker images

Expected result

The image list contains Nginx, Tomcat, Ubuntu, Jenkins, and SonarQube. Your initial output showed these repositories and tags:

Repository Tag

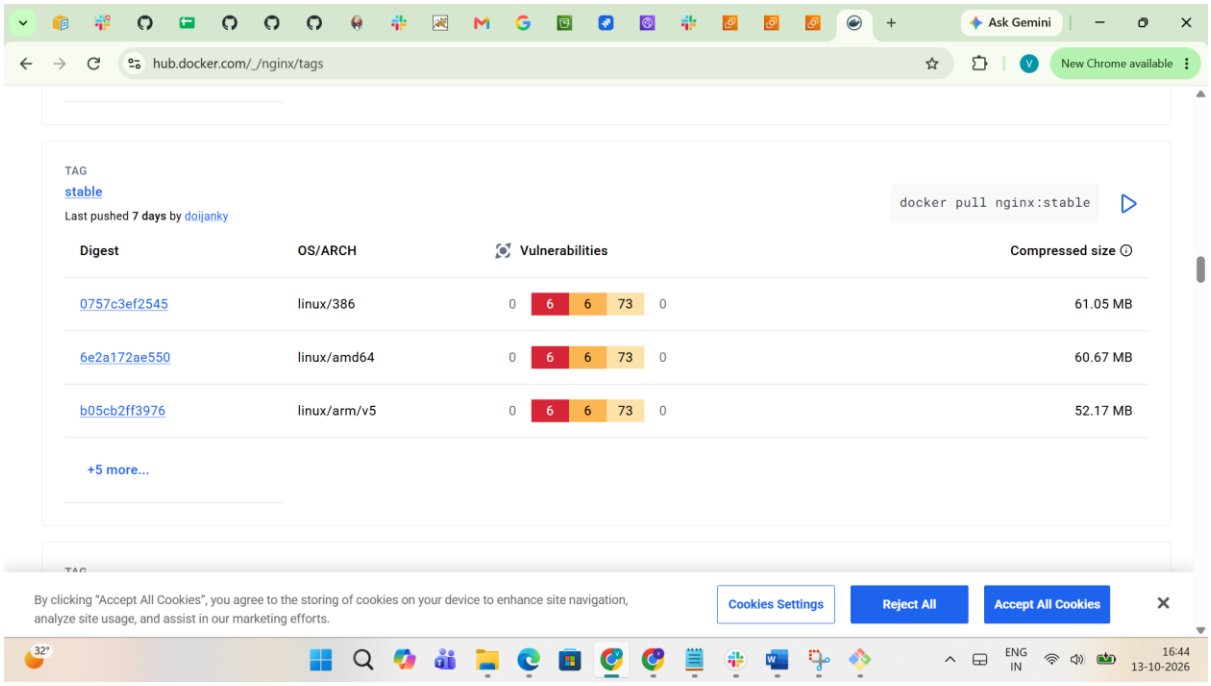
nginx stable

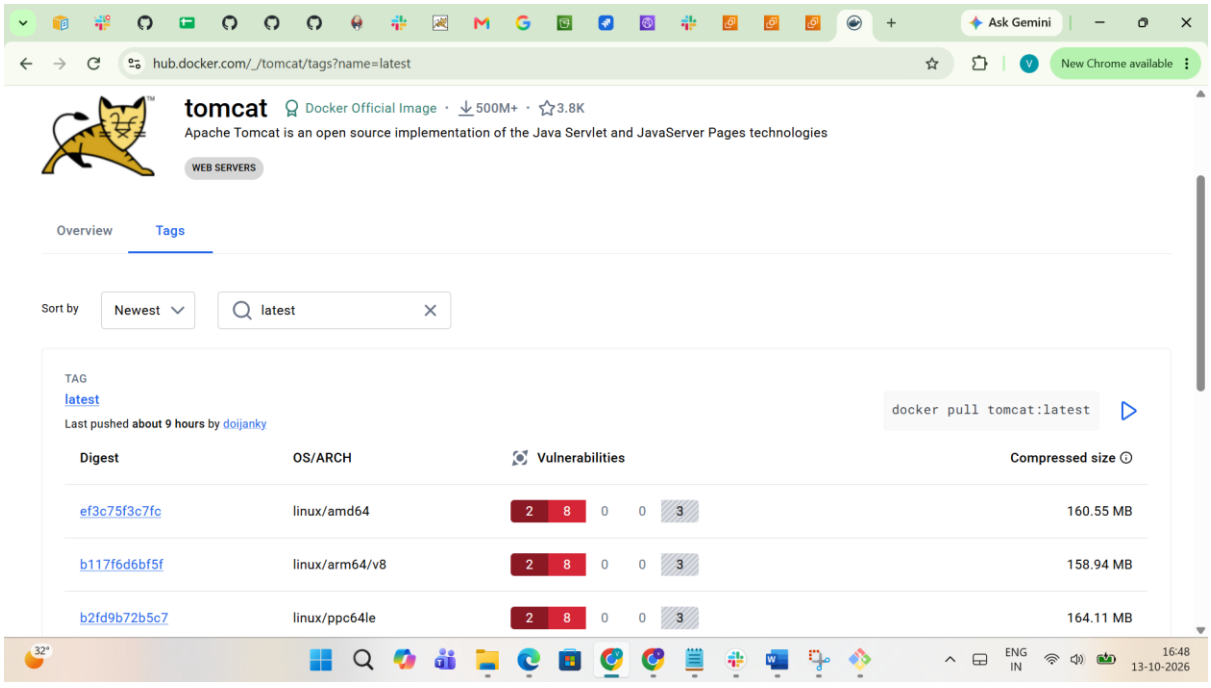
tomcat latest

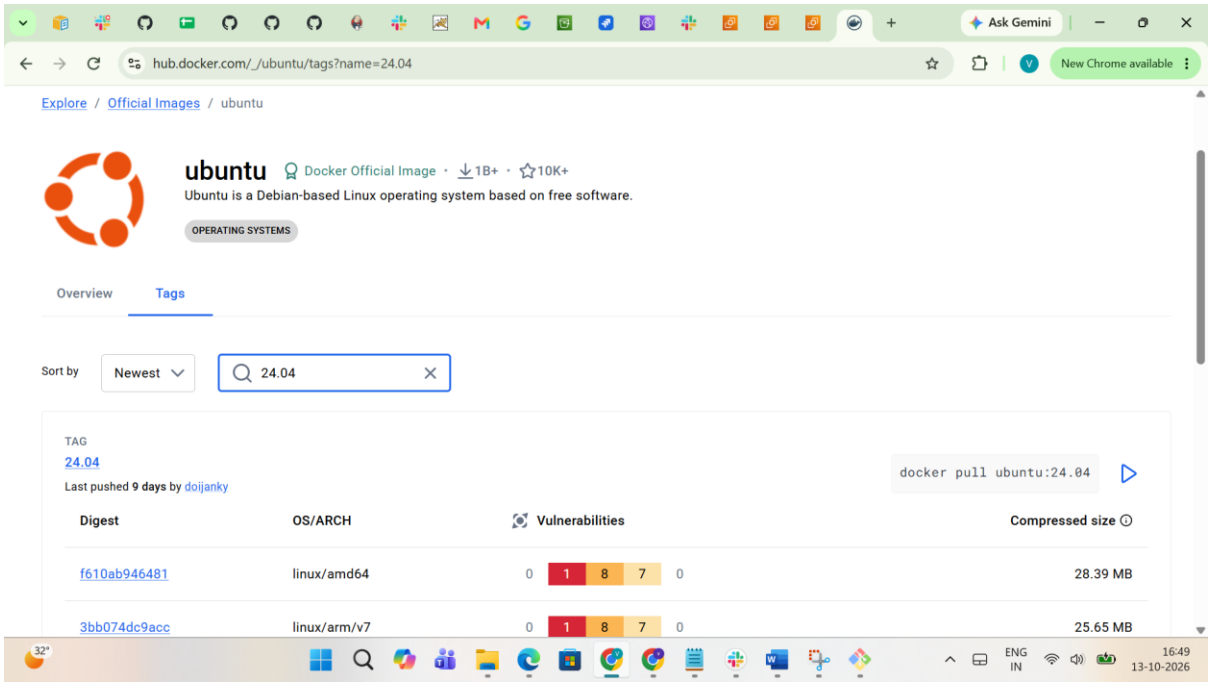
ubuntu 24.04

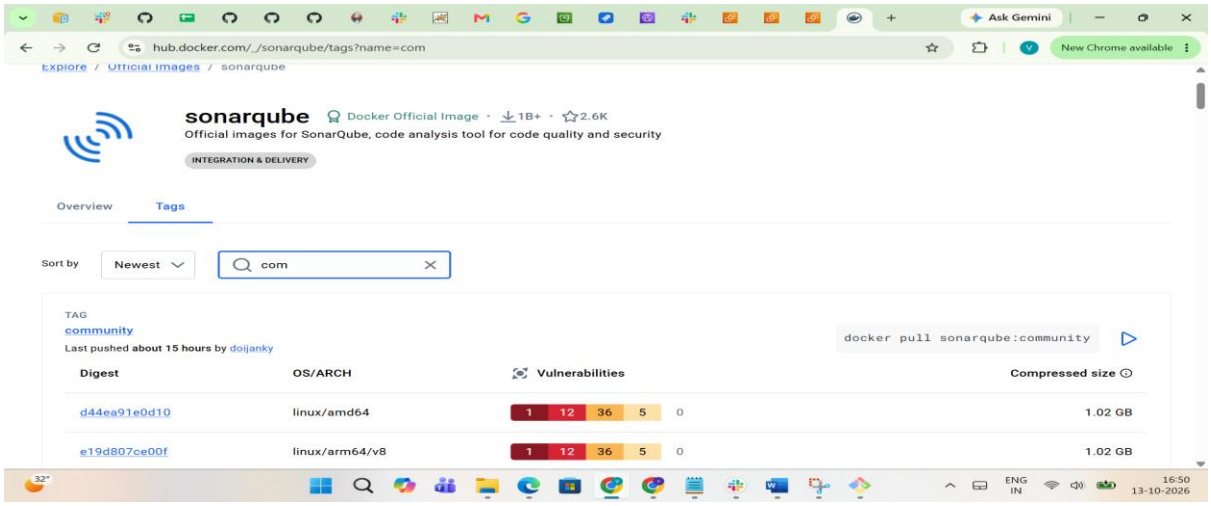
jenkins 2.60.3

sonarqube community

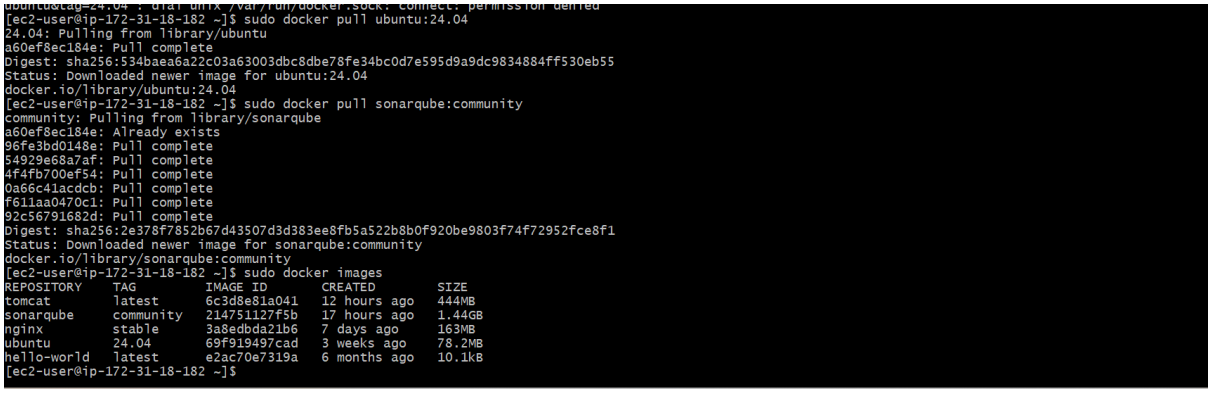









```
91738@IDEAPAD MINGW64 ~/Downloads
$ ssh -i "linux-testing.pem" ec2-user@ec2-18-208-217-254.compute-1.amazonaws.com
The authenticity of host 'ec2-18-208-217-254.compute-1.amazonaws.com (18.208.217.254)' can't be established.
ED25519 key fingerprint is SHA256:xh5UCl7apSvjzco61wk7hDTzzCIK4p/p7TtwEyJxx20.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-18-208-217-254.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Oct 13 10:54:31 2026 from 18.206.107.28
[ec2-user@ip-172-31-18-182 ~]$ sudo docker pull nginx:stable
stable: Pulling from library/nginx
ec5100c1e359: Pull complete
f4616bb1be1c: Pull complete
f1a6a629b824: Pull complete
3d774efa34e3: Pull complete
cb44c3fe542b: Pull complete
121907e00c1c: Pull complete
220a6ac36368: Pull complete
Digest: sha256:9bf97bd7714f5e24c1ccd545ecb9eb5435cb6d109c97cebb15e7e455e0239edb
Status: Downloaded newer image for nginx:stable
docker.io/library/nginx:stable
[ec2-user@ip-172-31-18-182 ~]$ sudo docker pull tomcat:latest
latest: Pulling from library/tomcat
4e07a0f12b2c: Pull complete
06ad70e463aa: Pull complete
5dec61325458: Pull complete
f39241fdc590: Pull complete
4f4fb700ef54: Pull complete
130739ca07b1: Pull complete
e59a999a9c9e: Pull complete
0b2769356b03: Pull complete
Digest: sha256:6974cddd46f09a10a0d4ffae6cd77f9122b3cf7de8f4b7c3305f35026accc81f
Status: Downloaded newer image for tomcat:latest
docker.io/library/tomcat:latest
```



4. Run nginx container and expose on port 81.

Implementation steps

1. Create and start the Nginx container sudo docker run -itd --name nginx-cont -p 81:80 nginx:stable Explanation:
   - -itd runs the container in the background.
   - --name nginx-cont assigns a name to the container.
   - -p 81:80 maps EC2 host port 81 to container port 80.
   - nginx:stable specifies the image.

2. Verify the running container sudo docker ps

The output should show nginx-container with a port mapping similar to: 0.0.0.0:81->80/tcp

3. Test Nginx from the EC2 terminal curl -I http://localhost:81

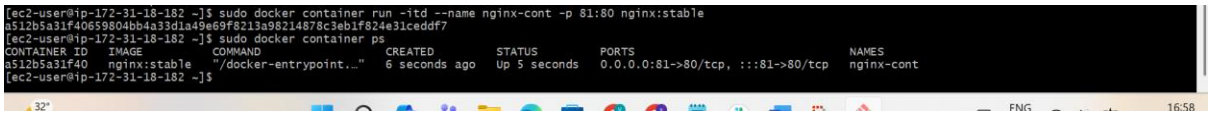
Expected response includes: HTTP/1.1 200 OK Server: nginx

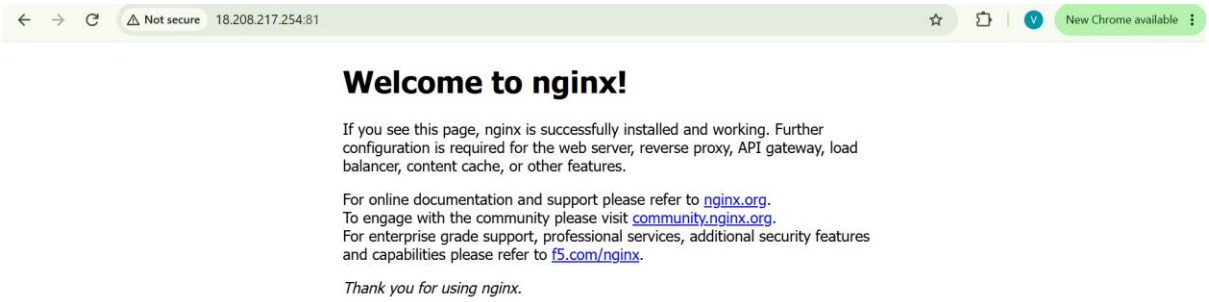
4. Configure the EC2 security group Ensure an inbound rule allows TCP port 81 from your IP address.

5. Open Nginx in a browser Use the following URL, replacing the placeholder with your EC2 public IPv4 address: http://YOUR-EC2-PUBLIC-IP:81

Expected result

The browser displays the Welcome to nginx! page.





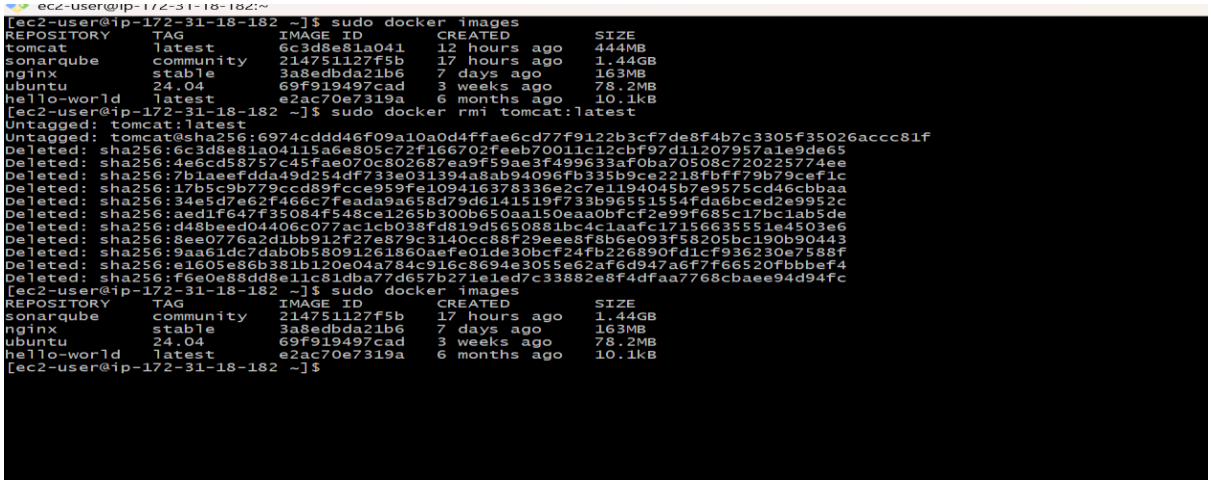
5. Delete the Apache tomcat image from local.

Implementation steps

1. Verify that the Tomcat image exists
   sudo docker images tomcat
   Your image used the latest tag.
2. Remove the image
   sudo docker rmi tomcat:latest
3. Verify the image list
   sudo docker images
   Confirm that the tomcat repository no longer appears.

Expected result

The local Tomcat image is removed successfully.



6. Inspect the Jenkins image, SonarQube image.

A. Inspect the Jenkins image

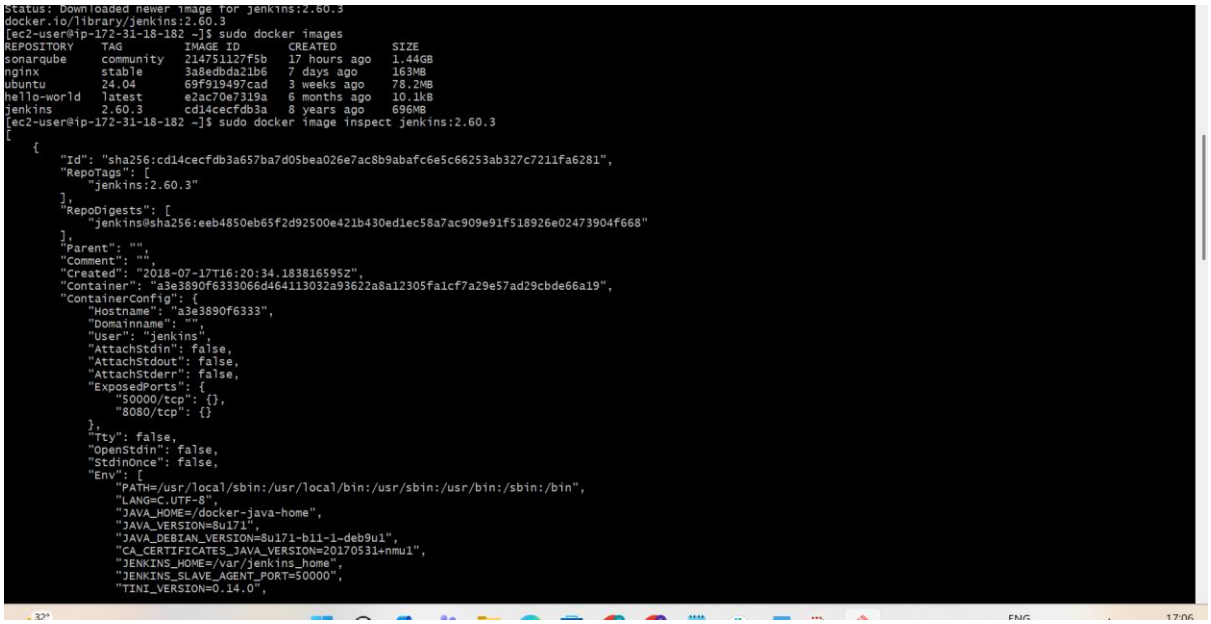
Your original Jenkins image was jenkins:2.60.3.

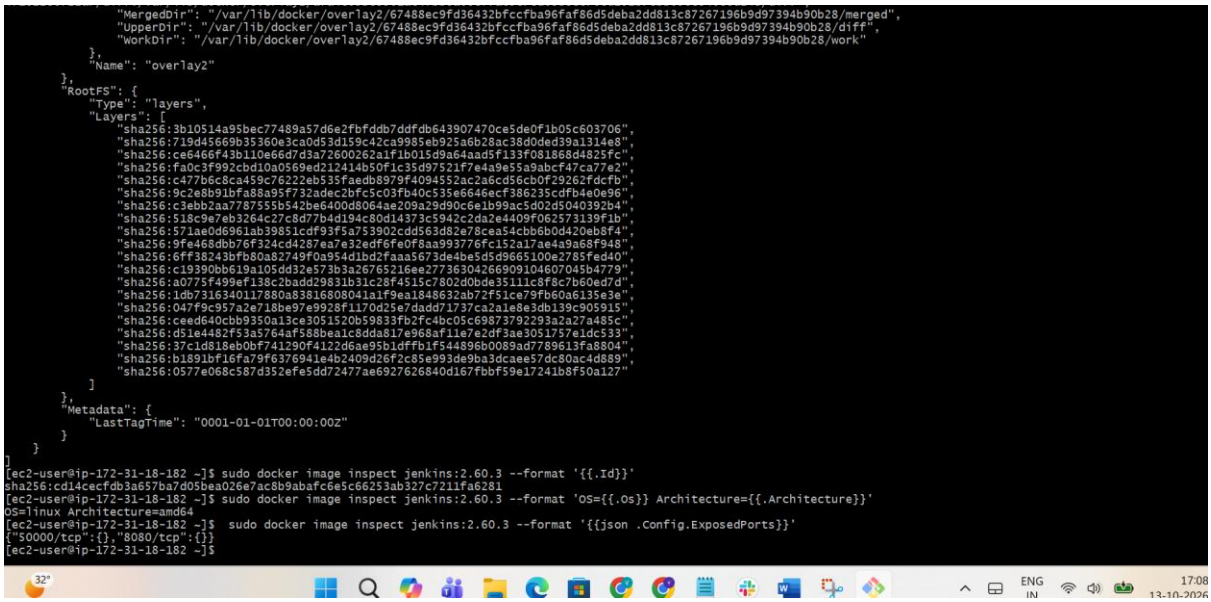
1. Display the complete image configuration sudo docker image inspect jenkins:2.60.3

2. Display the image ID sudo docker image inspect jenkins:2.60.3 --format '{{.Id}}'

3. Display OS and architecture sudo docker image inspect jenkins:2.60.3 --format 'OS={{.Os}} Architecture={{.Architecture}}'

4. Display exposed ports sudo docker image inspect jenkins:2.60.3 --format '{{json .Config.ExposedPorts}}'





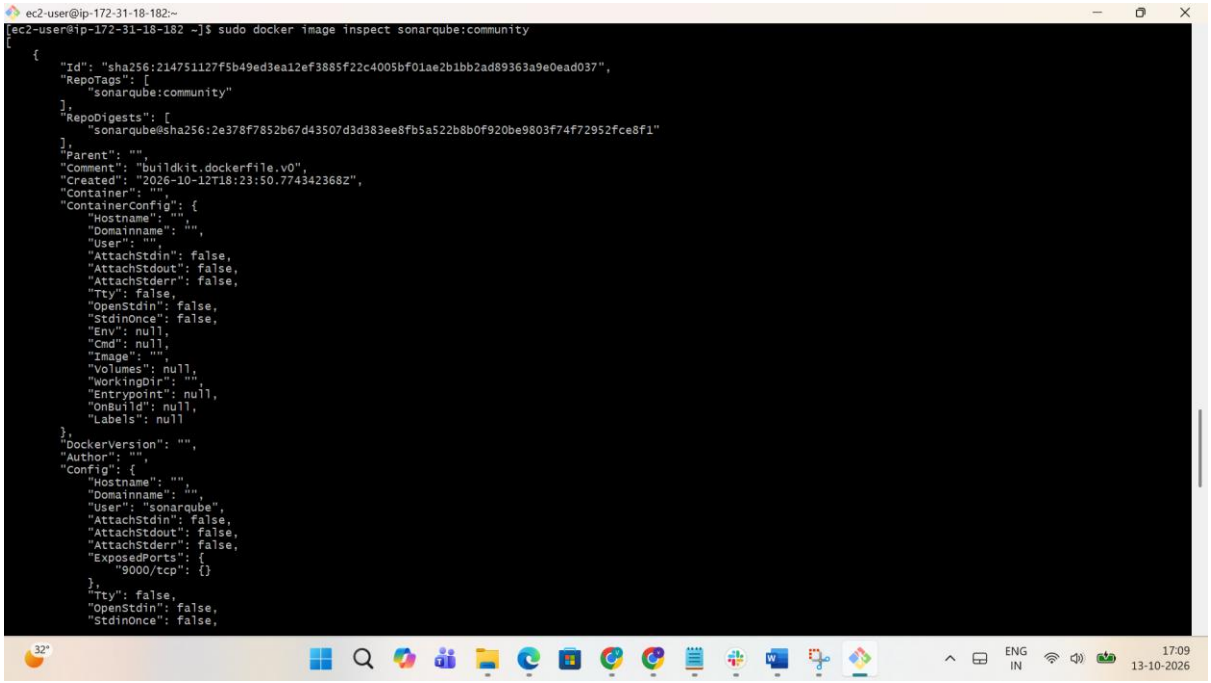
B. Inspect the SonarQube image

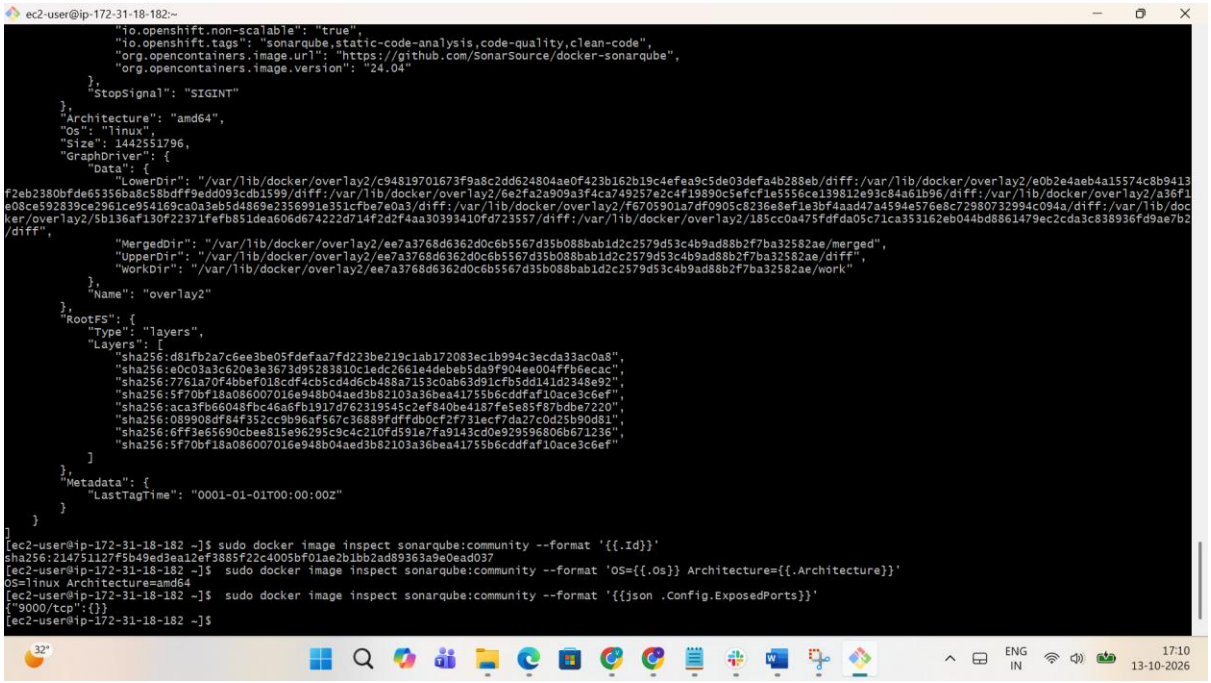
1. Display the complete image configuration sudo docker image inspect sonarqube:community

2. Display the image ID sudo docker image inspect sonarqube:community --format '{{.Id}}'

3. Display OS and architecture sudo docker image inspect sonarqube:community --format 'OS={{.Os}} Architecture={{.Architecture}}'

4. Display exposed ports sudo docker image inspect sonarqube:community --format '{{json .Config.ExposedPorts}}'





Expected result

Docker displays metadata for both images, including their IDs, platform information, and configured ports.

Command or field	Purpose
docker image inspect	Displays detailed image metadata
.Id	Shows the image identifier
.Os	Shows the operating system recorded in the image
.Architecture	Shows the CPU architecture
.Config.ExposedPorts	Shows ports documented by the image

7. Run Jenkins container and run one sample job.

Implementation steps

1. Pull the Jenkins image Download the maintained Jenkins image from Docker Hub.

sudo docker pull jenkins/jenkins:lts-jdk21

2. Create a Docker volume Create persistent storage for Jenkins data.

sudo docker volume create jenkins_home

Verify the volume: sudo docker volume ls

3. Run the Jenkins container

sudo docker run -d --name jenkins-container --restart unless-stopped -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk21

This command starts Jenkins in the background, maps port 8080 on EC2 to port 8080 inside the container, and stores Jenkins data in the persistent volume.

4. Verify the running container

sudo docker ps

Confirm that jenkins-container is running and port 8080 is mapped correctly.

5. Check Jenkins startup logs

sudo docker logs --tail 100 jenkins-container

Wait for Jenkins to finish starting.

6. Retrieve the initial administrator password

sudo docker exec jenkins-container cat /var/jenkins_home/secrets/initialAdminPassword

Copy the displayed password for the initial setup.

7. Access Jenkins through a web browser

• Open the AWS EC2 console and copy the instance's public IPv4 address.

• Ensure the EC2 security group allows inbound TCP port 8080 from your IP address.

• Open the following URL, replacing the placeholder with your actual public IP: http://YOUR-EC2-PUBLIC-IP:8080

• Enter the initial administrator password.

• Click Install suggested plugins.

• Create an administrator account and complete the setup.

8. Create a Freestyle project

Under Build Steps, select Add build step → Execute shell.

Enter the following shell script:

```
echo "Docker Jenkins Sample Job"

echo "Build Number: $BUILD_NUMBER"

echo "Job Name: $JOB_NAME"

date

whoami

pwd

hostname

echo "Build completed successfully!"
```

Click Save.

9. Execute the sample job

10.Click Console Output.

11. Verify that the output contains Build completed successfully! and the build status is successful.

Expected result

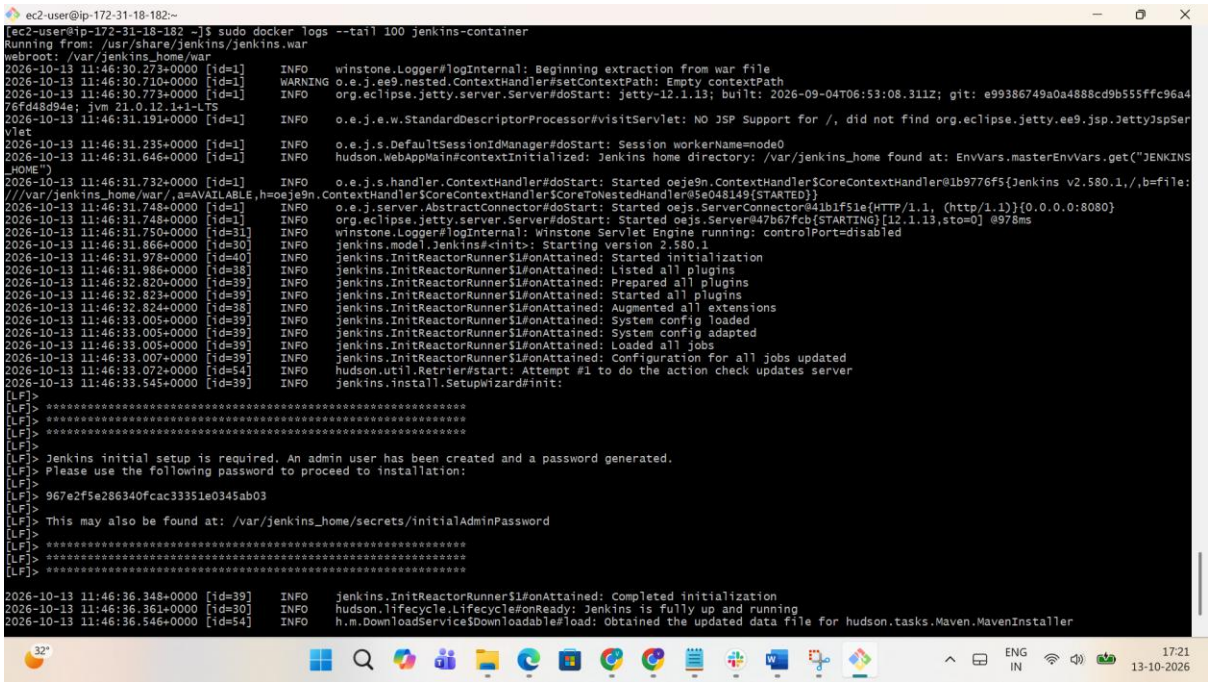
• Jenkins runs successfully inside a Docker container.

• The Jenkins web interface is accessible through port 8080.

• The Docker-Sample-Job Freestyle project executes successfully.

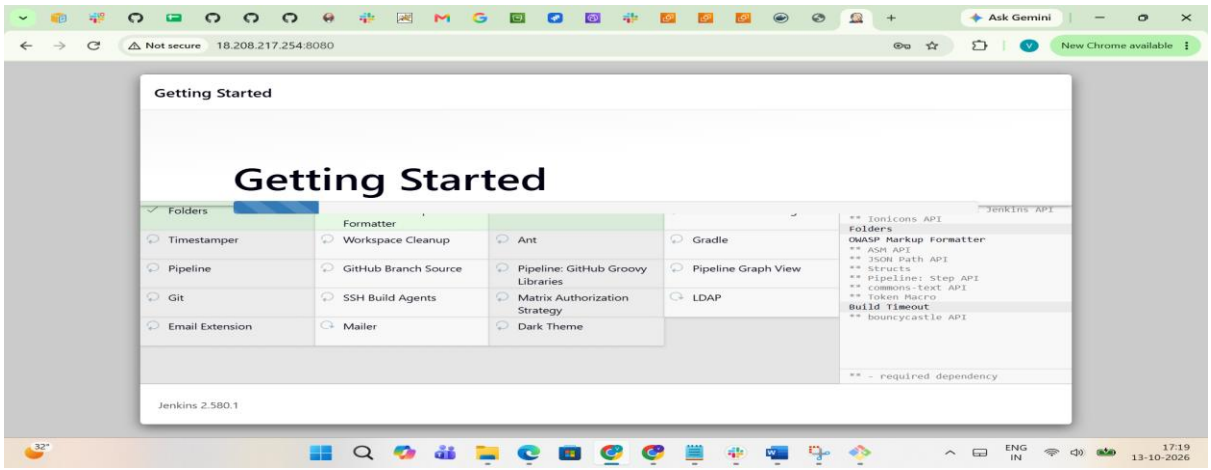
• The Console Output displays the build details and success message.

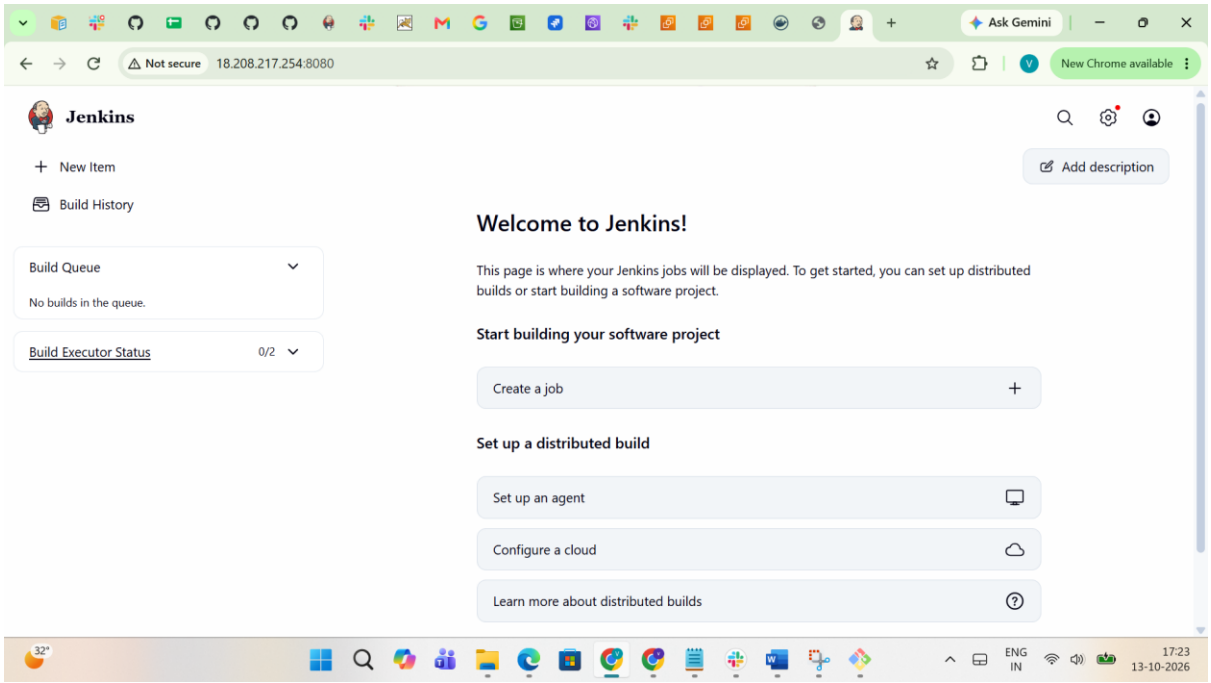
```
[ec2-user@ip-172-31-18-182 ~]$ sudo docker pull jenkins/jenkins:lts-jdk21
lts-jdk21: Pulling from jenkins/jenkins
6eefb2f5d3e9: Pull complete
408777b27aa9: Pull complete
810008851f3a: Pull complete
d89089c8a5a3: Pull complete
701ded2f9b33: Pull complete
968c36a3b2c5: Pull complete
3dd4a99993ab: Pull complete
a20a14499a3e: Pull complete
463a574793f8: Pull complete
f9e31fab50e8: Pull complete
95bc7464aaea: Pull complete
a91427fe0e3f: Pull complete
Digest: sha256:a660310e39ade10631f774bacd5219de767dfd08947ea5c32a739b5e5bb382c1
Status: Downloaded newer image for jenkins/jenkins:lts-jdk21
docker.io/jenkins/jenkins:lts-jdk21
[ec2-user@ip-172-31-18-182 ~]$ sudo docker volume create jenkins_home
jenkins_home
[ec2-user@ip-172-31-18-182 ~]$ sudo docker volume ls
DRIVER    VOLUME NAME
local     jenkins_home
[ec2-user@ip-172-31-18-182 ~]$ sudo docker run -itd --name jenkins-container --restart unless-stopped -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk
21
8a9108eba301c3c939c204e71a140a84391a2bcd31a7a24ac2f4fa5057569840
[ec2-user@ip-172-31-18-182 ~]$ sudo docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
8a9108eba301   jenkins/jenkins:lts-jdk21   "/usr/bin/tini -- /u…"   21 seconds ago   Up 20 seconds   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp, 50000/tcp   jenkins-containe
r
a512b5a31f40   nginx:stable             "/docker-entrypoint.…"   18 minutes ago   Up 18 minutes   0.0.0.0:81->80/tcp, :::81->80/tcp                      nginx-cont
[ec2-user@ip-172-31-18-182 ~]$
```

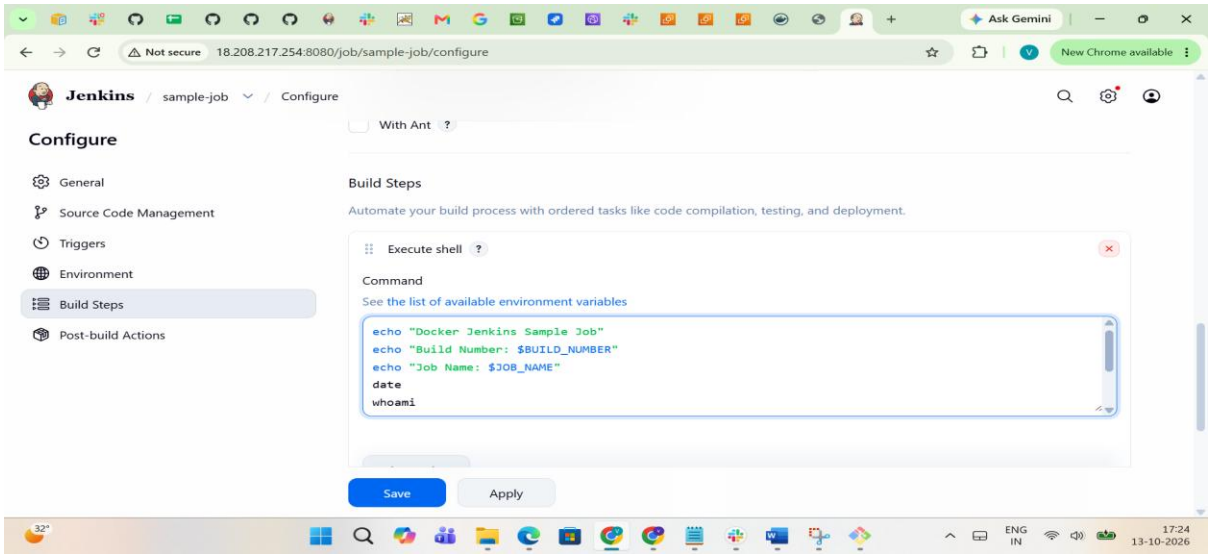


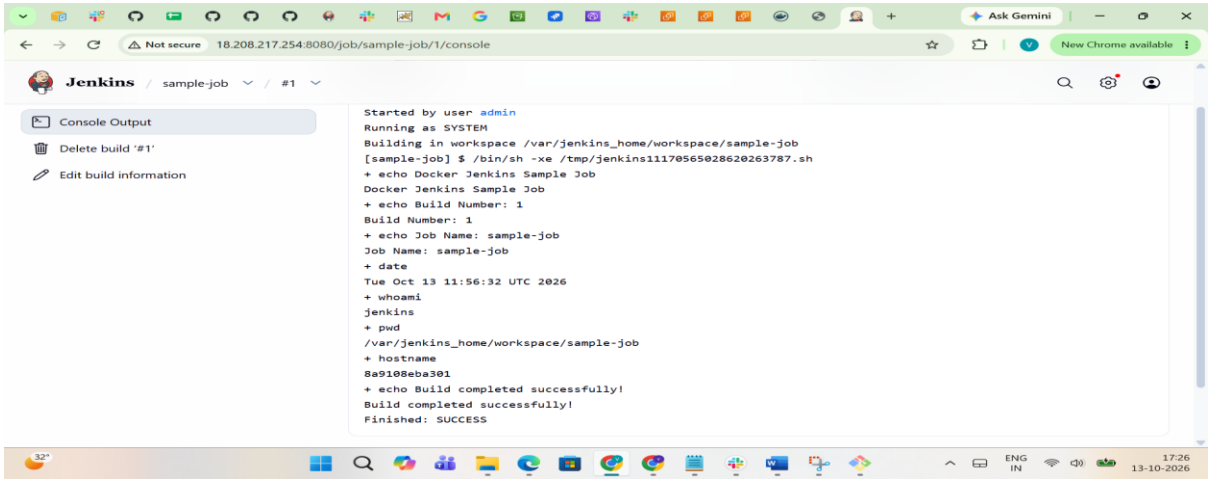

```
[LF]>
[LF]> 967e2f5e28634Ofcac33351e0345ab03
[LF]>
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]>
[LF]> ************************************************************
[LF]> ************************************************************
[LF]> ************************************************************

2026-10-13 11:46:36.348+0000 [id=39]    INFO    jenkins.InitReactorRunner$1#onAttained: Completed initialization
2026-10-13 11:46:36.361+0000 [id=30]    INFO    hudson.lifecycle.Lifecycle#onReady: Jenkins is fully up and running
2026-10-13 11:46:36.546+0000 [id=54]    INFO    h.m.DownloadService$Downloadable#load: Obtained the updated data file for hudson.tasks.Maven.MavenInstaller
2026-10-13 11:46:36.547+0000 [id=54]    INFO    hudson.util.Retrier#start: Performed the action check updates server successfully at the attempt #1
[ec2-user@ip-172-31-18-182 ~]$ sudo docker exec jenkins-container cat /var/jenkins_home/secrets/initialAdminPassword
967e2f5e28634Ofcac33351e0345ab03
[ec2-user@ip-172-31-18-182 ~]$
```




Not secure 18.208.217.254:8080
Jenkins
New Item
Build History
Build Queue
No builds in the queue.
Build Executor Status 0/2
Add description
Welcome to Jenkins!
This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.
Start building your software project
Create a job
Set up a distributed build
Set up an agent
Configure a cloud
Learn more about distributed builds


Not secure 18.208.217.254:8080/job/sample-job/configure
Jenkins / sample-job / Configure
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
With Ant
Build Steps
Automate your build process with ordered tasks like code compilation, testing, and deployment.
Execute shell
Command
See the list of available environment variables
echo "Docker Jenkins Sample Job"
echo "Build Number: $BUILD_NUMBER"
echo "Job Name: $JOB_NAME"
date
whoami
Save
Apply


Not secure 18.208.217.254:8080/job/sample-job/1/console
Jenkins / sample-job / #1
Console Output
Delete build '#1'
Edit build information
Started by user admin
Running as SYSTEM
Building in workspace /var/jenkins_home/workspace/sample-job
[sample-job] $ /bin/sh -xe /tmp/jenkins11170565028620263787.sh
+ echo Docker Jenkins Sample Job
Docker Jenkins Sample Job
+ echo Build Number: 1
Build Number: 1
+ echo Job Name: sample-job
Job Name: sample-job
+ date
Tue Oct 13 11:56:32 UTC 2026
+ whoami
jenkins
+ pwd
/var/jenkins_home/workspace/sample-job
+ hostname
8a9108eba301
+ echo Build completed successfully!
Build completed successfully!
Finished: SUCCESS